

AIGC检测 · 全文报告单

NO:CNKIAIGC2026FG_202605130106429

检测时间: 2026-05-14 15:35:54

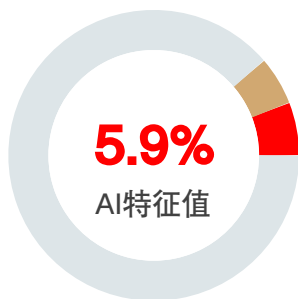
篇名: 王昌振毕业论文

作者: 王昌振

单位: 烟台大学

文件名: 王昌振毕业论文.docx

全文检测结果 知网AIGC检测 <https://cx.cnki.net>



AI特征值: 5.9%

AI特征字符数: 1147

总字符数: 19604

- AI特征显著 (计入AI特征字符数)
- AI特征疑似 (未计入AI特征字符数)
- 未标识部分

AIGC片段分布图

前部20%

AI特征值: 0.0%

AI特征字符数: 0

中部60%

AI特征值: 9.8%

AI特征字符数: 1147

后部20%

AI特征值: 0.0%

AI特征字符数: 0



分段检测结果

序号	AI特征值	AI特征字符数 / 章节(部分)字符数	章节(部分)名称
1	0.0%	0 / 2977	中英文摘要等
2	16.5%	649 / 3935	1 绪论
3	4.9%	498 / 10208	3 系统总体设计_第1部分
4	0.0%	0 / 2484	3 系统总体设计_第2部分

1. 中英文摘要等

AI特征值：0.0%

AI特征字符数 / 章节(部分)字符数：0 / 2977

片段指标列表

序号	片段名称	字符数	AI特征
1	片段1	361	疑似

原文内容

基于Spring AI的电网智能运维的设计与实现

指导教师贺利坚学历本科职称副教授所学专业计算机科学与技术

具体要求(主要内容、基本要求、主要参考资料等)：

主要内容：在Spring Boot框架上构建电网关键设备的监控与运维系统。一方面通过实时可视化看板集中展示关键设备运行状态，另一方面集成发电预测AI模型能够实现发电量的预测。使用Spring AI，利用RAG技术为系统注入了电网行业知识库，使得运维人员可以通过自然语言直接查询设备状态或一键生成标准化工单。整个系统以Spring Boot作为核心，通过RESTful API整合了Java后端业务逻辑、Python AI分析服务以及前端可视化界面，确保了预测预警与运维响应的高效协同。

基本要求：实现系统核心功能，达成高内聚、低耦合目标，方便开发及后期维护。

主要参考资料：Spring Boot、Vue、MySQL 、SpringAI、Python、等相关技术文档及教程。

进度安排：

(1) 选题：2025年11月21日前

(2) 开题：2025年12月15日前

(3) 中期检查：2026年3月27日前

(4) 答辩：2026年5月下旬-6月上旬

指导教师（签字）：

年月日

院（系）意见：

教学院长（主任）（签字）：

年月日

备注：

摘要

12.1%(361)

电网运维工作强调故障定位速度和处理闭环，但在实际业务中，设备台账、状态数据、告警记录和维修经验往往分散在不同系统或文档中。运维人员排查问题时既要查设备信息，又要翻阅手册和历史处理记录，信息切换成本较高。针对这一问题，本文结合毕业设计开发过程，设计并实现了一套基于 Spring AI 的电网智能运维平台。

平台以 RuoYi-Vue 为基础进行二次开发，后端采用 Spring Boot 3、Spring Security、MyBatis 和 Spring AI，前端采用 Vue 2、Element UI 与 ECharts，数据主要存储在 MySQL 中。系统实现了设备台账、状态监测、规则告警、RAG 知识问答、自然语言查库、工单流转、风电功率预测和站内消息等功能。

在智能问答部分，系统将运维文档上传后完成解析、清洗、分块和向量化保存，用户提问时先检索相关片段，再组织模型回答，以减少回答脱离资料的问题。风电预测部分通过 Java 调用 Python 服务完成 GRU 模型推理，并将预测结果返回前端展示和入库保存。

本文在本机部署环境下完成了模块功能、异常输入、权限边界和浏览器兼容性验证。测试结果显示，各主要流程能够正常运行，告警、工单、问答和预测模块之间可以形成基本联动，说明该系统对电网运维信息集中管理和智能辅助处理具有一定实践价值。

关键词：智能电网；运维管理；Spring AI；检索增强生成；风电功率预测

ABSTRACT

The efficiency of power-grid operation and maintenance is closely related to how quickly faults are located and handled. During the project investigation, I found that device ledgers, alarm records, maintenance experience and technical manuals are often kept in separate places. Engineers therefore have to switch between pages, databases and documents before making a decision. Based on this problem, this thesis designs and implements a Spring AI based intelligent operation and maintenance platform for power-grid scenarios.

The platform is developed on top of RuoYi-Vue. The back end uses Spring Boot 3, Spring Security, MyBatis and Spring AI, while the front end uses Vue 2, Element UI and ECharts. MySQL is used as the main database. The implemented functions include device management, status monitoring, rule-based alarms, RAG question answering, natural-language database query, maintenance work orders, wind-power prediction and internal message notification.

For the RAG function, uploaded maintenance documents are parsed, cleaned, split and stored as vectors before being used for retrieval-

based answers. For wind-power prediction, the Java service calls a Python prediction service, where a GRU model performs short-term sequence inference and returns the result to the business system.

The system was tested in a local deployment environment. The verification covered normal function flows, abnormal inputs, role permissions and browser compatibility. The results show that the main modules can run correctly and support a basic closed loop among alarms, work orders, intelligent question answering and prediction results.

Keywords: smart grid; operation and maintenance management; Spring AI; retrieval-augmented generation; wind power forecasting

2.1 绪论

AI特征值: 16.5%

AI特征字符数 / 章节(部分)字符数: 649 / 3935

片段指标列表

序号	片段名称	字符数	AI特征		
2	片段1	251	疑似		6.4%
3	片段2	649	显著		16.5%

原文内容

1 绪论

1.1 研究背景与意义

从本课题接触到的电网运维场景看，运维人员最常遇到的困难并不是单个功能缺失，而是信息分散带来的处理链条过长。设备基础信息、运行状态、告警记录、维修文档和人工经验各自独立保存时，故障定位往往需要依赖人工记忆和反复检索。随着设备数量增加和新能源接入比例提高，这种工作方式在响应速度、过程留痕和协同处理方面都会暴露出不足[14][15]。

从技术演进看，电网运维已经从定期人工巡检逐步转向在线采集和辅助决策[17]。在线监测能够把电压、电流、温度、功率等数据留下来，但这些数据如果仍然分散在不同页面和文档中，现场人员仍需要花时间判断“哪台设备、什么异常、该按哪条经验处理”。风电、光伏接入后，出力波动也会让状态判断更加复杂[3]。因此，本文把规则告警、知识检索和短期预测放入同一后台，重点解决数据看得到但不容易用起来的问题。

基于上述背景，本文选择将设备管理、状态监测、知识问答、工单处理和预测

分析放在同一平台中实现。这样设计的直接目的,是让设备状态变化、告警触发、经验查询和工单处理能够围绕同一设备编号展开,减少人为整理和重复录入。RAG 技术可用于把手册和故障经验转化为可检索知识[4],GRU 等时序模型则可为短期趋势分析提供辅助依据[5]。

本文完成的工作是一套面向电网运维业务的 Spring AI 应用系统。系统以 RuoYi-Vue 作为管理后台基础,结合 Spring Boot、Spring AI、MySQL、Redis、Vue 和 Python 预测服务,重点实现设备台账、规则告警、智能问答、自然语言查库、工单闭环和风电功率预测等模块,并通过测试验证其基本可用性。

1.2 国内外研究现状

现有电力运维研究一方面关注平台化建设,另一方面关注具体业务环节的智能化改造。工业互联网平台在设备接入和数据汇聚方面已有较多实践,国内电力行业的研究则更重视运维流程、人员角色和现场处置规则的结合[14][15]。这说明通用平台可以提供底座能力,但若真正服务电网运维,还需要围绕设备、告警、工单和知识资料进行二次设计。

技术框架方面,Java Web 项目中使用 Spring Boot 可以减少基础配置工作,RuoYi-Vue 已经提供了菜单、角色、权限、代码生成等后台管理能力[7][10],比较适合作为毕业设计的工程底座。Spring AI 的价值在于把模型调用、向量检索和对话接口封装为较统一的组件[8],DashScope 等平台则提供了可直接调用的在线模型服务[9]。本文选择这些技术,并不是单纯追求新技术堆叠,而是希望在现有管理系统结构中验证 AI 功能的接入方式。

设备状态监测和故障诊断是运维平台必须解决的基础环节。已有研究将机器学习用于异常识别、状态分类和故障检测,高芳等对电气设备故障检测算法的适用情况进行了整理[12]。这些成果说明算法能够辅助判断异常,但落实到本系统时,还需要把算法结果与设备编号、告警规则、接口返回和页面操作结合起来,否则很难形成完整业务流程。

新能源功率预测研究中,GRU、LSTM 和注意力机制等模型应用较多。刘茂强等将 GRU 用于风电功率超短期预测[2],张胜等对新能源功率预测模型和场景进行了综述[3],也有研究尝试使用 LSTM 与注意力机制改进序列建模效果[13]。结合本课题的开发周期和部署条件,本文没有继续增加复杂模型结构,而是采用较容易训练、接口也便于封装的 GRU 作为预测服务核心。

在知识问答方面,RAG 通过先检索资料再组织回答的方式,为行业文档问答提供了可借鉴思路[4]。GRU 的门控结构研究也为时序预测模型设计提供了理论基础[5]。综合来看,故障检测、功率预测和知识问答等方向已有不少单点成果,但如何在一个 Java 企业级系统中把这些能力与权限、工单、消息和数据库查询流程结合起来,仍有进一步实践空间。

1.3 研究内容与研究意义

1.3.1 研究内容

本课题的开发目标来自实际开发过程中的几个痛点：页面要能跑通，数据要能查到，处理记录要能追溯。围绕这些要求，我先把设备台账、状态记录、告警、知识文档、工单和预测结果之间的关系画出来，再逐步落到数据库表、后端接口和前端页面。系统接入 Spring AI 对话、RAG 检索、NL2SQL 查询和 GRU 预测服务后，仍保持模块化单体结构，这样在毕业设计周期内更便于联调、演示和排查问题。

实现过程中，系统大致分为数据、业务和智能三个部分。MySQL 负责保存设备、告警、工单、对话和预测记录，Redis 主要用于缓存与告警去重辅助；业务层沿用 Spring Boot 分层写法，通过 Service 组织流程并用事务保证关键数据一致；智能层集中在 spring-ai 模块，负责大模型对话、RAG 问答和自然语言查库。风电预测部分单独由 Python 服务完成 GRU 推理，Java 后端只负责传参、接收结果、校验并写回数据库。

1.3.2 研究意义

本研究的意义主要体现在以下三个层面。

(1) 工程实践层面。立足电网运维真实流程，本文将设备管理、规则告警、知识问答、工单流转与预测分析整合到统一平台，推动跨环节数据贯通与流程协同。基于现有实现，在一定程度上能够减少信息传递损耗，并提升故障响应效率与运维闭环能力

(2) 技术融合层面。在同一业务系统内协同集成 Spring Boot、Spring AI、RAG 与时序预测模型，构建了由“规则约束—数据分析—智能辅助”共同驱动的技术框架。传统管理系统引入 AI 能力并非简单叠加功能，而是可以形成可复用的工程化方案。

(3) 应用推广层面。在保障核心业务稳定运行的前提下，系统通过超时控制、异常回退与服务降级机制提升运行速度，降低外部 AI 服务响应时间长对业务连续性的影响。进一步看，这一实现为电力、能源及其他工业运维场景的扩展应用提供了可借鉴的样例与方法基础。

1.4 论文结构

全文共六章：第一章介绍研究背景与国内外研究现状；第二章进行系统需求分析；第三章阐述系统总体设计；第四章详述各模块详细设计与实现；第五章给出测试环境、测试用例与结果分析；第六章总结全文并展望后续工作。

2 系统需求分析

本章先把系统要解决的问题边界说明清楚，再从业务功能、非功能约束和核心技术三个角度展开分析。这样处理的目的是，避免后续设计只停留在功能罗列上，而是把设备、告警、知识库、工单和预测之间的关系提前梳理清楚。

2.1 业务功能需求

6.4%(251)

16.5%(649)

本系统面向智能电网运维场景，核心目标是将设备状态、告警事件、知识服务、工单处理与预测结果整合到统一平台，形成可追踪、可协同的运维闭环。结合实习中了解的项目需求，业务功能需求如下。

(1) 设备与状态数据管理需求。系统需要维护设备编号、设备类型、安装位置、运行状态等基础信息，并保存电压、电流、温度、功率等状态记录。页面应支持按设备、类型和时间条件查询，查询结果需要与后续告警、工单和预测功能共用。

(2) 规则告警联动需求。系统需支持基于设备状态与规则配置的自动告警触发，记录告警级别、触发时间、关联设备和处理状态；同时具备消息去重能力。

(3) 智能问答与知识检索需求。系统需要支持运维文档上传和入库，对文本进行分块与向量化处理。运维人员输入问题后，系统应先检索相关资料，再调用模型生成回答，使回答尽量来源于已有手册或经验记录。

(4) 自然语言查库需求。系统需支持将自然语言问题转换为 SQL 并受控执行，返回结构化查询结果；

(5) 工单管理需求。系统应支持工单创建、分派、处理、审核和完成等状态流转，并保存关联设备、告警来源、负责人和处理说明，方便后续追溯。

(6) 风电功率预测需求。系统应能够调用 GRU 预测服务，展示短期功率变化曲线，并将预测时间、功率值、风险等级等信息保存到数据库，供后续对比查看。

(7) 消息通知协同需求。告警生成、工单状态变化、系统公告等事件需要形成站内消息，并支持未读数量、已读状态和消息列表查询，便于运维人员及时了解处理进度。

2.2 非功能与安全需求

非功能需求主要来自实际使用时的稳定性要求。设备列表、告警详情和工单处理属于基础业务，即使外部模型接口变慢或失败，也不能影响这些页面继续使用。因此，AI 相关接口需要有超时控制、错误提示和降级处理；普通业务代码则继续采用 RuoYi 中常见的 Controller、Service、Mapper 结构，方便后续阅读和维护。

系统实现采用分层和模块化结合的方式。设备管理模块提供统一的设备编号，状态记录、告警、工单和预测结果都通过该编号建立关联；AI 对话、RAG 和自然语言查库集中放在 spring-ai 模块中，避免智能功能散落到普通业务代码里。这样划分后，某个模块出现问题时可以单独排查，也方便后续替换模型或扩展功能。

安全与隐私保护方面，系统没有让模型直接操作数据库写入流程，而是由后端统一接收、校验和执行。用户登录、菜单权限和按钮权限沿用 RuoYi 的 Spring Security、JWT 与 RBAC 体系；工单审核、规则配置等关键操作写入日志，便于出现问题后回看操作来源。

2.3 术语说明

为避免后文概念混用，这里对几个术语作简要说明。本文中的 RAG 是指“先查

资料、再组织回答”的问答流程；自然语言查库是指把用户问题转换为只读 SQL，并在后端校验后返回查询结果；工单闭环则表示从发现异常、创建任务、人员处理到结果归档的一整套记录过程。

3.3 系统总体设计_第1部分

AI特征值: 4.9%

AI特征字符数 / 章节(部分)字符数: 498 / 10208

片段指标列表

序号	片段名称	字符数	AI特征		
4	片段1	498	显著		4.9%
5	片段2	448	疑似		4.4%

原文内容

3 系统总体设计

本章从系统架构、技术选型、数据库设计和业务流程四个方面对系统进行总体设计，为第四章的详细实现提供基础。

3.1 逻辑架构与技术选型

系统逻辑架构采用前后端分离设计，整体分为前端展示层、安全控制层、业务服务层、智能能力层和数据层。用户通过浏览器访问系统，前端基于 Vue2 与 Element UI 实现页面交互，通过 Axios 调用后端接口，AI 对话场景使用 EventSource 实现流式返回，预测结果通过 ECharts 展示。系统逻辑架构图如图 3.1.1。

后端以 Spring Boot 为核心，按多模块组织业务能力：ruoyi-admin 负责接口入口，ruoyi-system 负责核心业务逻辑，ruoyi-framework 提供安全、日志和缓存等基础能力，ruoyi-quartz 负责定时任务。系统在该层实现设备管理、状态监测、规则告警、工单流转和消息通知等功能。

系统安全控制放在请求进入业务处理之前完成。用户访问接口时，Spring Security 先校验登录状态，JWT 用于识别当前用户，RBAC 决定其可见菜单和可执行按钮。对于规则修改、工单审核等操作，系统会记录操作日志，方便后期追踪。

智能功能集中放在 spring-ai 模块中，普通业务模块只通过接口调用其能力。问答部分接入 DashScope 模型，知识库问答使用 RAG 检索，数据库问答在生成 SQL 后还要经过后端安全检查。风电预测没有直接放在 Java 进程内运行，而是通过 FastAPI 调用 Python 中的 GRU 模型，便于模型训练和业务系统解耦。

数据层以 MySQL 为主要存储，用于保存设备、状态、告警、工单、对话、预测

和站内消息等数据；Redis 主要承担热点数据缓存、会话加速和告警去重辅助。这样的设计把业务数据持久化和高频访问优化分开处理，使系统在功能完整的同时保持较好的扩展余地。

图3.1.1 系统逻辑架构图

系统技术栈如表3.1.2所示。总体来看，本研究在技术选型上综合考虑了开发效率、运行性能、生态成熟度与团队技术储备，形成了面向电网运维场景、具备可实现性、可维护性与可扩展性的工程方案。

表3.1.2系统技术栈选型	
层次	技术选型说明
后端框架	Spring Boot 3.2 + Spring AI 企业级应用开发
前端框架	Vue 2 + Element UI 单页应用与组件化开发
数据持久层	MySQL 8 + MyBatis 结构化数据存储与ORM
缓存层	Redis 6（可选） 热点数据缓存加速
AI 引擎	通义大模型 DashScope 语言理解与生成能力
预测引擎	TensorFlow + Python GRU时序预测模型
向量存储	SimpleVectorStore 文档嵌入向量检索

系统技术选型主要是参考目前大多数小公司采用的技术，方案成熟稳定。后端采用 Spring Boot 3.2.x 与 Java 21，利用线程池并发处理能力；Spring AI 作为统一的 AI 集成框架，提供 ChatClient、VectorStore 等抽象接口。数据持久层采用 MyBatis 并结合 PageHelper 分页插件，兼顾手写 SQL 的可控性与动态 SQL 组装的灵活性。前端选用 Vue 2 与 Element UI，可直接复用 RuoYi-Vue 现有的代码生成模板和权限指令体系，从而缩短功能落地周期。

在开发工具方面，后端使用 IntelliJ IDEA，前端使用 VS Code；项目构建与依赖管理由 Maven 负责，并通过 spring-ai-bom 统一约束 Spring AI 及相关依赖版本。版本控制采用 Git，代码托管于 Git 仓库；接口调试使用 Postman，数据库可视化管理使用 Navicat。上述工具在 Java 企业级开发中已有较充分的实践验证，工具生态稳定、团队上手成本较低，能够较好支撑项目开发效率与质量控制。

3.2 数据库和业务流程

系统数据库采用 MySQL 8，字符集统一为 utf8mb4。表命名字段采用蛇形命名，主要业务表保留时间字段。库表关系以业务字段关联为主，逻辑上保持主从一致；该设计能够兼顾读写性能与维护成本。核心业务表包括设备台账表 eq_device、设备状态表 eq_device_status、规则表 eq_device_rule、告警表 eq_alert_record、运维工单表 ai_maintenance_form、AI 对话记录表 ai_chat_record、预测结果表 eq_prediction，以及设备预测绑定表 eq_wind_forecast_bind、站内消息表 sys_user_message 等辅助表。

eq_device 作为设备主数据中心，是状态监测、告警联动、工单管理与预测分

析等模块的共同基础。

表3.2 eq_device（设备台账表）结构

字段名类型约束说明

device_id BIGINT PRIMARY KEY 设备编号，自增主键

device_name VARCHAR(100) NOT NULL 设备名称

device_type VARCHAR(50) INDEX 设备类型编码

specification VARCHAR(100) 规格型号

location VARCHAR(200) 安装位置

commission_date DATE 投运日期

manufacturer VARCHAR(100) 制造商名称

device_status VARCHAR(20) DEFAULT 'normal' 设备状态

create_time DATETIME NOT NULL 创建时间

update_time DATETIME 更新时间

eq_device_status 表记录设备运行状态的采样数据，字段通过 device_id 与设备台账对应。实际使用时，页面趋势图、阈值规则、告警判断和预测前置数据都会读取这张表。为了后续按设备和时间范围检索，表中重点保留采集时间、指标名称、指标值和状态描述等字段。

表3.3 eq_device_status（设备状态表）结构

字段名类型约束说明

status_id BIGINT PRIMARY KEY 状态编号，自增主键

device_id BIGINT FOREIGN KEY 关联eq_device.device_id

temperature DECIMAL(10,2) 设备温度（℃）

voltage DECIMAL(10,2) 运行电压（kV）

current DECIMAL(10,2) 运行电流（A）

power_factor DECIMAL(5,4) 功率因数

status_desc VARCHAR(500) 状态描述信息

record_time DATETIME NOT NULL 数据记录时间

告警记录表 eq_alert_record 用于保存系统触发的告警事件及其处理状态，是运维人员识别异常、跟踪处置进度的重要依据。

表3.4 eq_alert（告警记录表）结构

字段名类型约束说明

alert_id BIGINT PRIMARY KEY 告警编号，自增主键

device_id BIGINT INDEX 关联设备编号

alert_type VARCHAR(50) NOT NULL 告警类型编码

alert_level VARCHAR(20) NOT NULL 告警级别：一般/紧急/严重

alert_content TEXT 告警内容描述

trigger_time DATETIME NOT NULL 告警触发时间
confirm_status VARCHAR(20) DEFAULT 'pending' 确认状态
handler VARCHAR(50) 处理人用户名
handle_time DATETIME 处理时间
handle_result VARCHAR(500) 处理结果

ai_maintenance_form 表保存工单编号、关联设备、问题描述、负责人、处理状态、处理意见和时间节点等信息。告警确认后生成工单，人工排查也可以直接创建工单。通过这些字段，系统能够看到某次异常是否已经分派、处理到哪一步以及最终处理结论是什么。

表3.5 ai_maintenance_form（运维工单表）结构

字段名 类型 约束 说明

form_id BIGINT PRIMARY KEY 工单编号，自增主键
device_id BIGINT INDEX 关联设备编号
title VARCHAR(200) NOT NULL 工单标题
content TEXT 工单详细内容
ai_summary VARCHAR(500) AI生成工单摘要
status VARCHAR(20) DEFAULT 'pending' 工单状态
create_by VARCHAR(50) NOT NULL 创建人
create_time DATETIME NOT NULL 创建时间
handler VARCHAR(50) 处理人
finish_time DATETIME 完成时间

ai_chat_record 表保存用户问题、模型回复、会话编号和时间等信息。该表的作用不是简单记录聊天内容，而是方便用户恢复上下文，也方便后续对问答过程进行复盘。

表3.6 ai_chat_record（AI对话记录表）结构

字段名 类型 约束 说明

id BIGINT PRIMARY KEY 记录编号，自增主键
session_id VARCHAR(100) INDEX 会话编号
question TEXT NOT NULL 用户提问内容
answer TEXT AI回答内容
mode VARCHAR(20) DEFAULT 'free' 对话模式
create_time DATETIME NOT NULL 创建时间

eq_prediction 表保存预测时间、预测功率、风险等级、置信度和关联设备等信息。预测结果写入该表后，前端不仅可以展示本次 24 小时曲线，也可以在后续查询历史预测记录，和实际运行数据进行对比分析。

表3.7 eq_prediction（风电功率预测表）结构

字段名类型约束说明

id BIGINT PRIMARY KEY 预测编号，自增主键

device_id BIGINT INDEX 关联设备编号

prediction_time DATETIME NOT NULL 预测时间点

actual_power DECIMAL(10,2) 实际功率 (kW)

predicted_power DECIMAL(10,2) NOT NULL 预测功率 (kW)

step INT 预测步长

create_time DATETIME NOT NULL 创建时间

图3.2 系统核心数据库E-R图

系统业务流程可分为三类。第一类是状态监测与告警：状态数据写入后，规则服务根据阈值和条件组合判断是否生成告警，并通过站内消息提醒相关人员。第二类是智能问答：用户在前端选择自由对话、知识库问答或数据库查询模式，后端分别进入模型对话、RAG 检索或 SQL 校验执行流程。第三类是功率预测：后端向 Python 服务发送预测请求，接收 GRU 模型结果后完成页面展示和数据保存。

4 系统详细设计与实现

本章按照系统功能模块说明详细设计与实现，重点写清楚后端 Controller、Service、Mapper 的分工，前端页面与接口的调用关系，以及 AI、工单、预测等模块在工程中的落地方式。

4.1 设备多维数据与状态管理

本模块是平台的数据基础，负责设备台账和运行状态的统一管理，服务于后续告警、工单和预测功能。

系统使用 eq_device 保存设备基础信息，使用 eq_device_status 保存设备状态历史。一个设备在不同时间会产生多条状态记录，因此可以在页面上查看同一设备的变化过程，也可以把这些记录作为告警判断和趋势展示的数据来源。

后端接口按照 RuoYi 的写法组织。EqDeviceController 负责接收 /equipment/device 下的新增、编辑、删除和分页查询请求；EqDeviceServiceImpl 处理设备业务规则；EqDeviceStatusServiceImpl 负责状态写入、更新和历史查询。状态列表支持设备、时间范围和指标条件筛选，查询结果按时间倒序展示。

数据访问层使用 MyBatis 编写 SQL，分页由 PageHelper 处理。设备列表支持名称模糊查询、类型筛选和时间范围筛选；导入导出复用 Excel 工具。导入时先检查必填项、字段格式和业务约束，确认无误后再写入数据库，导出时按照当前查询条件生成文件。

性能上采用 MySQL + Redis 协同方案。状态写入时先落库再更新缓存；状态读取优先走缓存，未命中回源数据库并回填。缓存键按 eq:status:{deviceId} 统一命名并设置过期时间。设备删除等多表操作通过事务控制，保证失败回滚，避免脏数据。

4.9%(498)

前端页面包含设备列表和详情视图。列表页支持分页与组合查询，详情页展示基础信息、运行参数和状态历史。状态趋势通过图表展示，并可按时间区间查看。该模块最终实现了设备数据与状态数据的一体化管理，为平台其他模块提供稳定数据支撑。

4.2 规则评估与告警联动

规则评估与告警联动模块负责把状态数据变化转化为可处理的告警事件。实现时将流程拆成触发和计算两部分：IEqDeviceRuleTriggerService 接收状态变更并启动评估，IEqDeviceRuleEvaluationService 负责阈值比较、条件组合和时间窗口判断。这样做可以使入口控制和规则计算相互独立，后续扩展规则类型时影响范围较小。

表4.1 告警规则配置示例

规则名称 条件表达式 告警级别 说明

高温告警 `temperature > 85` 严重 设备温度超上限

电压越限 `voltage < 95 OR voltage > 115` 紧急 电压超出正常运行范围

功率因数低 `power_factor < 0.85` 一般 功率因数偏低需关注

电流过载 `current > 120% I_n` 严重 电流超额定值120%

4.3 对话、RAG 与自然语言查库

本系统把自由对话、知识库问答、数据库问答和历史会话放在同一组页面中，减少运维人员在多个入口之间切换。后端的 AI 能力集中在 spring-ai 模块：AiDeomController 处理普通对话和流式输出，RagController 处理文档入库和检索问答，DatabaseQueryController 处理自然语言查库，ChatHistoryController 保存会话记录并支持历史回看。

知识库构建从文档上传开始。TXT 文档直接读取文本，PDF 文档使用 PDFBox 提取内容，DOCX 文档按段落解析。系统会先过滤空行、页眉页脚等无效内容，再使用 TokenTextSplitter 按约 500 Token 切分，并保留约 50 Token 重叠。切分后的片段写入 SimpleVectorStore，同时把文件名、大小、上传时间等元数据保存到 ai_rag_document 表，便于后续管理和重新索引。

知识问答采用先检索后回答的方式。用户输入问题后，系统把问题向量化并检索相似片段，默认取 Top-3 作为上下文，再连同原问题发送给模型。这样回答内容会更多围绕已上传资料展开，避免模型只凭通用知识给出过于笼统的建议。

自然语言查库采用“生成 SQL—安全检查—受控执行”的流程。模型生成 SQL 后，后端只允许 SELECT 查询，并拦截删除、修改、授权等危险关键词，同时限制返回条数。校验通过后再由 JdbcTemplate 执行，结果以结构化数据返回前端；如果问题过于复杂，系统会提示用户拆分条件后重新查询。

模型调用由 Spring AI 的 ChatClient 和 StreamingChatClient 完成，通义模型名称、温度和最大 Token 等参数写入统一配置。知识库问答使用较低温度，主

要保证回答稳定；自由对话可以适当提高温度，使表达更自然。参数集中管理后，后续调试不需要频繁修改业务代码。

会话记录按会话编号写入 `ai_chat_record`，内容包括用户输入、模型输出、模式类型和时间信息。用户刷新页面或重新进入系统后，可以根据历史列表恢复上一次对话，这对运维交接和问题复盘比较有帮助。

前端对话页采用类似即时通讯的布局。流式回答通过 `EventSource` 连接 `/springai/chat/stream` 接口，服务端按 `SSE` 推送增量文本，前端边接收边渲染，收到结束标记后关闭连接。`NL2SQL` 查询结果则使用表格展示，并附带简短说明，方便非开发人员理解查询结果。

本模块的安全处理分为输入、执行和异常三个方面：输入侧限制长度并过滤明显敏感内容；执行侧对 `SQL` 类型和关键词进行白名单校验；异常侧针对模型超时、密钥失效、知识库为空等情况返回明确提示，必要时回退到普通对话。通过这些处理，`AI` 功能不会直接影响核心业务稳定性。

4. 4%(448)

图4.1 RAG知识库问答流程示意图

4.4 运维表单与预测服务集成

运维表单模块主要对应告警后的处理流程。系统通过 `ai_maintenance_form` 记录工单编号、设备、问题描述、负责人、当前状态和处理意见。工单既可以由运维人员手动创建，也可以在告警确认后生成，用来把“发现问题—分派人员—填写处理结果—审核归档”的过程保存下来。

后端接口按照创建、分派、处理、审核和完成几个动作拆分。每次状态变化都会更新工单记录，并同步写入操作日志。为了避免状态被随意跳转，服务层对流转顺序进行限制，例如未分派的工单不能直接进入完成状态，已完成的工单不允许继续修改处理内容。

前端页面提供工单列表、详情查看和状态操作入口。运维人员可以按设备、负责人、状态和时间范围筛选工单；进入详情页后，可以查看关联设备信息、告警来源、处理说明和历史流转记录。这样的设计便于班组交接，也方便后期追溯某次异常的处理过程。

风电功率预测采用 `Java` 后端调用 `Python` 服务的方式实现。页面提交设备和预测时间范围后，`Java` 侧组装请求并通过 `HTTP` 访问 `FastAPI` 接口；`Python` 侧加载已经训练好的 `GRU` 模型，完成 24 小时功率预测，并返回功率值、置信度和风险等级等字段。

预测结果返回后，后端先做格式校验，再写入 `eq_prediction` 表。前端使用 `ECharts` 绘制预测曲线，并结合风险等级进行颜色区分。这样既可以直观看到未来一段时间的功率变化，也可以把预测结果作为后续运维判断的辅助信息。

跨语言调用中最容易出现的问题是服务未启动、响应超时和返回字段不完整。针对这些情况，后端设置了请求超时和结果校验：`Python` 服务不可用时直接返回可

读提示，页面不会一直等待；返回数据缺少必要字段时不写入数据库，并提示用户重新检查预测服务。

工单模块和预测模块都以设备主数据为入口。工单更关注已经发生的异常如何处理，预测更关注未来一段时间的趋势变化。二者通过统一设备编号关联后，用户在设备详情中既能看到历史工单，也能查看预测记录，从而把事后处置和事前研判放到同一条信息链上。

整体来看，本节实现了运维事件处理和预测结果管理两个能力：前者解决告警后的任务流转和过程留痕问题，后者为设备状态研判提供趋势参考。二者与前文的告警、消息和知识问答模块配合，构成了较完整的运维辅助流程。

图4.2 运维工单全生命周期流转流程

4.5 消息推送、权限与工程化实现

本节主要说明平台在消息通知、权限控制和工程治理方面的实现。

消息触达采用 WebSocket 与 SSE 分工处理：告警和工单状态变化通过 WebSocket 推送，AI 对话内容通过 SSE 流式返回。前端导航栏设置消息中心，展示未读数量和消息列表；站内消息统一由 /system/user-message 接口管理，并对重复提醒做了简单去重。

在权限控制上，系统采用 RBAC 机制，前后端共同校验。后端通过 @PreAuthorize 做接口鉴权，前端通过 v-hasPermi 控制按钮和功能显示。管理员与普通运维人员权限分离，权限标识按“模块:功能:操作”命名，能够满足日常运维场景下的访问边界管理。

前端基于 Vue2、Element UI、Vue Router、Vuex 和 Axios 组织页面与请求。删除等风险操作增加二次确认，SSE 连接在页面销毁时主动关闭。后端使用统一异常处理和事务控制关键流程，并为外部 AI 调用设置超时和降级，减少外部服务波动对普通业务的影响。

系统还做了基础工程化处理，包括跨域白名单、安全响应头、操作日志、审计留痕和接口文档管理。开发阶段保留接口文档便于联调，生产环境可关闭文档入口。前端代码按照 ESLint 和 Prettier 进行格式约束，README 用于记录启动和部署说明。

整体来看，本模块主要完成三项支撑能力：消息能够及时到达，权限能够限制不同角色的操作范围，关键过程能够通过日志和记录进行追踪。

图4.3 RBAC权限模型示意图

5 系统测试

5.1 测试环境与功能验证

本章对系统各功能模块进行了测试验证。测试在单机环境下开展，覆盖功能测试、异常场景测试与兼容性验证。

测试工作在本机开发环境中完成。后端运行环境为 Windows 10、JDK 21 和

Spring Boot 3.2.x，数据库使用 MySQL 8，Redis 用于缓存和部分去重场景；前端分别在 Edge 与 Chrome 中访问。由于没有接入真实现场设备，本章测试重点放在功能链路是否能跑通、异常输入是否有提示、权限边界是否生效。

测试时按模块逐项操作，而不是只看页面是否能打开。设备管理部分检查新增、修改、查询、删除和导入导出；AI 部分检查普通对话、知识库问答、数据库问答以及空输入、超长输入等情况；工单部分检查状态流转和消息触发；预测部分检查 Java 请求 Python 服务、结果回传和图表显示；权限部分检查管理员和普通用户看到的按钮是否不同。

本轮共整理 15 个核心用例，基本覆盖论文中实现的主要业务流程。测试结果显示，设备台账、告警触发、工单状态变更、智能问答、自然语言查库、预测结果返回和权限控制都能完成预期操作。尤其是 RAG 问答在上传资料后能够检索到相关片段，预测模块也能返回 24 小时数据并生成连续曲线。

表5.1 系统功能测试用例及结果

测试模块测试用例预期结果实际结果

设备管理添加新设备设备信息保存成功，列表正确显示通过

设备管理按类型筛选设备返回对应类型的设备列表通过

设备管理导出Excel 生成含查询结果的Excel文件通过

告警管理阈值触发告警超阈值数据写入后自动生成告警通过

告警管理告警确认操作确认后告警状态更新为已确认通过

AI问答知识库模式提问答案包含上传文档的相关信息通过

AI问答自由对话模式模型返回合理的自然语言回答通过

AI问答数据库查询模式正确生成SQL并返回查询结果通过

工单管理创建运维工单工单生成并在列表中显示通过

工单管理工单流转审批状态接待处理-处理中-已完成流转通过

工单管理 AI生成摘要自动提取关键信息生成工单摘要通过

功率预测执行预测返回预测曲线并与实际数据对比通过

功率预测未绑定设备预测使用全局配置数据回退预测通过

消息通知告警消息推送相关用户收到对应告警通知通过

权限控制越权访问API 未授权请求返回403状态码通过

设备管理测试使用管理员账号完成。测试中先新增一条设备记录，再修改设备名称、类型和安装位置，保存后返回列表检查字段是否同步；组合查询分别按名称、类型和时间范围执行。删除设备时系统会给出确认提示，Excel 导入会拦截缺少必填项或格式错误的的数据。

告警管理测试采用手动写入异常状态数据的方式进行。状态值超过规则阈值后，系统能够生成告警记录并显示告警级别、设备信息和触发时间。对于同一设备短时间内连续出现的相同异常，去重逻辑能够减少重复提醒，测试结果与规则配置基

本一致。

4.3 系统总体设计_第2部分

AI特征值: 0.0%

AI特征字符数 / 章节(部分)字符数: 0 / 2484

片段指标列表

序号	片段名称	字符数	AI特征
----	------	-----	------

原文内容

AI 问答测试分为普通对话、知识库问答和数据库问答。普通对话主要检查流式返回是否连续；知识库问答先上传运维资料，再提问与资料相关的问题，观察回答是否引用检索片段；数据库问答重点检查生成 SQL 后的安全校验和表格化结果展示。

工单和预测模块测试关注流程是否闭合。工单从创建、分派、处理到审核的状态变化能够按顺序完成；预测功能可以把页面参数发送到 Python 服务，并将返回结果展示在 ECharts 曲线上。测试过程中，未出现页面长时间卡死或主流程被外部服务阻断的情况。

- 图5.1.1 设备信息修改功能测试截图
- 图5.1.2 设备信息新增功能测试截图
- 图5.1.3 基础对话功能测试截图
- 图5.1.4 知识库问答功能测试截图
- 图5.1.5 数据库问答功能测试截图
- 图5.1.6 发电预测功能测试截图
- 图5.1.7 运维表单生成功能测试截图
- 图5.1.8 消息推送功能测试截图
- 图5.1.9 告警管理功能测试截图

5.2 性能、用例与兼容性

本章对系统各功能模块进行了测试验证，测试在单机环境完成，覆盖功能测试、异常场景测试和兼容性测试。

兼容性和边界测试同样在本机环境完成。后端仍使用 JDK 21、Spring Boot 3.2.x、MySQL 8 和 Redis，前端主要对 Edge、Chrome 两个浏览器进行验证。测试方法以黑盒测试为主，分别覆盖正常操作、空值输入、错误参数、重复提交和权限不足等场景。

从执行结果看，15 个核心用例均可完成。设备管理、规则告警、AI 问答、RAG

检索、NL2SQL 查询、工单流转、消息通知、预测服务调用和权限控制都达到预期效果。预测图表能够连续显示 24 小时数据，普通用户与管理员在可见按钮和接口权限上也能体现差异。

整体来看，系统在当前环境下运行稳定，核心业务链路可用。

5.3 局限性

AI 服务可用性仍是当前系统的短板。问答能力主要依赖在线模型接口，网络较差或服务不可用时连续性会受到影响。后续可以引入本地轻量模型或离线知识库作为兜底，形成在线模型优先、本地能力补充的双通道方案。

在数据形态支持方面，平台目前以文本数据处理为主，对图像等非结构化数据利用不足。后续可扩展视觉分析能力，面向红外热像图、仪表盘图像等场景引入图像识别与异常检测方法，推动系统由“文本智能”向“多模态智能”升级。

需要说明的是，当前系统主要使用仿真数据和历史样本完成验证，还没有接入真实传感器和现场采集终端。因此，本文结论更适合作为原型系统和工程集成验证。后续如果能够接入真实电表、变压器和采集网关，再建立“实时采集—在线分析—闭环处置”的验证链路，系统的工程价值会更充分。

6 总结与展望

本次毕业设计从一个具体问题出发：如何把设备数据、告警处理、运维知识和预测结果组织到同一个系统中。开发过程中，我完成了后端接口、数据库表、前端页面、RAG 问答、NL2SQL 查询、站内消息以及 Java 调用 Python 预测服务等工作。系统最终形成了“状态采集—规则判断—消息提醒—工单处理—知识辅助—结果记录”的基本闭环。

这个项目对我最大的收获，是把课堂中学到的 Java Web、数据库、前端和机器学习知识放到同一个工程问题中验证。很多问题只有在联调时才会暴露出来，例如接口字段不一致、流式输出中断、预测服务返回格式变化、权限按钮和后端鉴权不一致等。解决这些问题的过程，也让我更理解工程开发中的细节和耐心。

通过本课题，我认识到 AI 在软件系统中的价值并不只是生成文字，而是可以作为业务流程的一部分参与查询、解释、检索和辅助判断。对于后端开发而言，工作重点也不仅是完成增删改查，还包括如何设计可靠的边界、如何限制模型输出风险、如何把 AI 能力嵌入已有业务流程。

智能体和大模型技术仍在快速发展，但在电网运维这类场景中，稳定性、可追溯性和安全边界仍然非常重要。本文实现的系统还只是一个毕业设计版本，更多地体现了功能验证和工程集成思路，距离真实生产环境仍需要大量数据接入、压力测试和现场规则校准。

后续工作可以从三个方面继续完善：第一，接入真实采集终端和更多设备类型，使状态数据来源更贴近现场；第二，引入本地模型或离线知识库作为网络异常时的兜底方案；第三，扩展多模态分析能力，支持对仪表盘图片、红外热像图等资料

进行识别和辅助判断。

总体来看，本课题完成了预期功能，也让我经历了一次较完整的需求分析、系统设计、编码实现和测试验证过程。后续我会继续围绕人工智能与工业互联网方向学习，把本次项目中积累的经验用于更多真实问题。

致谢

论文写到这里就要结束了。回头看从选题、开发到写作和修改的整个过程，我很清楚这份成果不是我一个人完成的。一路上老师、同学、朋友和家人都给了我很多帮助和支持，在这里我向大家表示真诚感谢。

我首先要感谢我的指导老师贺利坚教授。无论是研究方向、技术路线，还是论文结构和细节修改，老师都给了我很多耐心、细致的指导，让我受益很大。

我也感谢烟台大学计算机与控制工程学院的各位老师。大学四年的学习中，老师们教给我的专业知识和思考方法，为我完成这个课题打下了坚实基础。

同时感谢济南真正科技有限公司在我 Java 实习期间提供的机会。通过实习，我接触了真实项目开发流程，对后端设计、接口规范、代码管理和团队协作有了更深理解，这段经历对本课题帮助很大。

同时感谢通义 DashScope、Spring AI、RuoYi-Vue 等项目和技术文档提供的参考。开发过程中，我通过阅读接口说明、调试示例代码和排查报错逐步完成模型调用、RAG 检索和后台权限集成，也因此对开源生态和工程文档的重要性有了更直接的体会。

最后最感谢我的家人。感谢你们一直以来的理解、包容和支持，在我压力大、状态不好的时候给我鼓励和力量，让我能安心完成论文。

论文完成不是终点，而是新的开始。接下来我会继续学习、认真实践，把学到的知识用到实际问题中。再次感谢所有帮助过我的人。

说明:

- 1、支持中、英文内容检测；
- 2、 $\text{AI特征值} = \text{AI特征字符数} / \text{总字符数}$ ；
- 3、红色代表AI特征显著部分，计入AI特征字符数；
- 4、棕色代表AI特征疑似部分，未计入AI特征字符数；
- 5、检测结果仅供参考，最终判定是否存在学术不端行为时，需结合人工复核、机构审查以及具体学术政策的综合应用进行审慎判断。



cx.cnki.net